Compito 8.2 APA

Fabio Nicolosi s7592267@studenti.unige.it

May 2026

1 Introduzione

Immaginiamo di avere due dispositivi: il computer A di Alice e il computer B di Bob. All'interno di essi sono memorizzati due file, che denomineremo rispettivamente x e y. Supponiamo che la dimensione di entrambi i file sia dell'ordine del Gigabit e che si voglia verificare se l'uguaglianza $x = y$ sia soddisfatta. L'approccio più intuitivo, seppur largamente inefficiente, consisterebbe nel trasmettere l'intero file a a Bob, affinché quest'ultimo possa verificarne l'esatta corrispondenza con il proprio file y. Tuttavia, come anticipato, questa procedura non è ottimale in quanto richiederebbe un consumo di banda eccessivo per la trasmissione dei dati, considerando che un file da 1 Gigabit è composto da 2^{30} bit.

2 Fondamenti teorici

Al fine di comprendere l'algoritmo risolutivo per il problema esposto nell'introduzione, è necessario introdurre alcuni concetti teorici fondamentali.

2.1 Numerosità asintotica dei numeri primi

Indichiamo con $\pi(x)$ il numero di numeri primi minori o uguali a un dato $x \in \mathbb{R}^+$. Il teorema sulla numerosità asintotica dei numeri primi afferma che:

$$\lim_{x\to\infty} \frac{\pi(x)}{x/\ln x} = 1$$

Inoltre, per il corretto funzionamento dell'algoritmo, è utile richiamare una semplice proprietà legata alla scomposizione in fattori: dato un numero intero $n < 2^l$ (con $l \in \mathbb{N}^+$), il numero dei suoi fattori primi è al più l.

2.2 Impronta digitale (*Fingerprinting*)

Sia p un numero primo scelto all'interno dell'insieme $\{2, \ldots, l^2\}$ e sia $n \in \mathbb{N}$ un numero tale che $n < 2^l$. Viene definita impronta digitale (o fingerprint) di n il risultato della seguente operazione modulare:

$$f(n) = n \pmod{p}$$

3 Algoritmo

```python
def GetP () -> int:
    length  = len(arrPrim)
    return arrPrim[random.randint(0, length)]

def MCStringEqualityVerifer(a:int, b:int, p:int):
    fa = a % p
    fb = b % p

    return ((abs (fa-fb) % p) == 0)
```

Codice Python

Possiamo subito applicare un concetto appena introdotto, ovvero l'impronta digitale (fingerprint) di un numero, per risolvere il problema posto nell'introduzione.Consideriamo i file x di Alice e y di Bob. Invece di trattarli come semplici sequenze di dati, interpretiamoli come grandi numeri interi:

- Sia a il numero intero corrispondente al file x di Alice.
- Sia b il numero intero corrispondente al file y di Bob.

Poiché entrambi i file sono formati da l bit, sappiamo per certo che i numeri interi associati saranno strettamente minori di 2^l.
A questo punto, il computer di Alice seleziona uniformemente a caso un numero primo $p \in \{2, \ldots, l^2\}$ e calcola l'impronta digitale del proprio file, ossia $f(a) = a \pmod{p}$. Una volta terminato il calcolo, Alice trasmette a Bob sia l'impronta ottenuta $f(a)$ che il numero primo p estratto casualmente.
Ricevuti questi dati, Bob utilizza lo stesso numero primo p per calcolare l'impronta del proprio file, ottenendo $f(b) = b \pmod{p}$. Ora Bob dispone di entrambe le impronte. Basterà verificare che il valore assoluto della loro differenza sia congruente a zero modulo p, ovvero valutare se $|f(a) - f(b)| \equiv 0 \pmod{p}$. Se questa condizione è verificata, esiste un'alta probabilità che i due file originali siano uguali ; se invece la condizione non è verificata, si ha la certezza matematica che i due file siano diversi.

3.1 Affidabilità dell'algoritmo

Poiché il numero primo $p \in \{2, \ldots, l^2\}$ viene campionato in modo casuale, esiste una probabilità di errore (falso positivo) per cui l'algoritmo indichi i file x e y come probabilmente uguali anche quando, in realtà, sono diversi. Chiamiamo questa probabilità $\mathcal{P}(l)$. Essa è stimata come:

$$\mathcal{P}(l) \approx \frac{2 \ln l}{l}$$

Come si evince dalla formula, per file di grandi dimensioni (quindi con l molto grande), tale probabilità risulta essere estremamente bassa.

4 Dati

```
Decimale letto in fileAlice.bin: 715827882
Decimale letto in fileBob.bin: 715827882

-------- Inizio MCStringEqualityVerifer --------

array di numeri primi:  [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61,
                         67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137,
                         139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211,
                         223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283,
                         293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379,
                         383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461,
                         463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563,
                         569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643,
                         647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739,
                         743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 821, 823, 827, 829,
                         839, 853, 857, 859, 863, 877, 881, 883, 887]

Dimensione array: 154
Numero p random: 557

Esiti per 1 volte :
        1: I file sono uguali

-------- Fine MCStringEqualityVerifer --------
```

Come si evince dall'output riportato, avendo fornito in input due file identici (entrambi corrispondenti al valore decimale 715827882), l'algoritmo non rileva differenze nelle loro impronte digitali calcolate con il modulo 557 e, coerentemente, classifica gli input come probabilmente uguali.

4.1 Tentativo di difficoltà

```
Decimale letto in fileAlice.bin: 1002154666
Decimale letto in fileBob.bin: 715827882

-------- Inizio MCStringEqualityVerifer --------

array di numeri primi:  [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61,
                         67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137,
                         139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211,
                         223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283,
                         293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379,
                         383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461,
                         463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563,
                         569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643,
                         647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739,
                         743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 821, 823, 827, 829,
                         839, 853, 857, 859, 863, 877, 881, 883, 887]

Dimensione array: 154
Numero p random: 89

Esiti per 10000 volte :
        1: I file NON sono uguali
        .
        .
        .
        10000: I file NON sono uguali
Tentativo con 521:

-------- Fine MCStringEqualityVerifer --------

I numeri che hanno generato un falso positivo sono:  [257, 2, 17]
```

Per valutare concretamente la probabilità di fallimento dell'algoritmo, ho eseguito una simulazione su due file distinti, aventi rispettivamente valore decimale $a = 1002154666$ e $b = 715827882$. L'operazione di verifica è stata iterata 1000 volte, campionando ad ogni ciclo un numero primo p in modo casuale dall'insieme di riferimento. L'esperimento ha rilevato alcune occorrenze in cui il verificatore ha classificato i file come "probabilmente uguali", generando dei veri e propri falsi positivi. Analizzando i dati di log, emerge che tali collisioni si sono verificate esclusivamente in corrispondenza dell'estrazione dei numeri primi 2, 17 e 257.

4.2 Calcolo probabilità

Per determinare i casi possibili, possiamo stimare il numero di numeri primi presenti nell'intervallo $\{2, \ldots, 900\}$ utilizzando il Teorema sulla numerosità asintotica dei numeri primi:

$$\pi(900) = \frac{900}{\ln 900} \approx \frac{900}{6} \approx 150$$

Come si evince dal calcolo, la stima teorica restituisce un valore di circa 150, a fronte dei 154 numeri primi effettivamente individuati dall'algoritmo (con uno scarto di 4 elementi).

Avendo appurato che i casi totali esatti sono 154, possiamo calcolare la probabilità che il numero primo da noi selezionato venga scelto identicamente in un'altra relazione. Assumendo una distribuzione di probabilità uniforme, essa è pari a:

$$\mathcal{P} = \frac{1}{154} \approx 0.65\%$$